

From Persistent Memory to Mission Working Sets:

An Authority-Aware Context Lifecycle Architecture for Agentic AI

Prasad Deshpande

Independent Researcher – Agentic AI Systems and Enterprise Architecture

ORCID: [0009-0008-1344-6585](https://orcid.org/0009-0008-1344-6585)

Email: er.prasad.deshpande@gmail.com

Research & Architecture: prasaddeshpande.com

v1.2.0 Public Implementation Update

1 October 2026

Abstract

Long-horizon AI agents accumulate conversation history, tool schemas, observations, retrieved evidence, execution traces, and durable memory. A common implementation pattern allows some or all of that accumulated state to remain computationally active on subsequent model calls. This creates a systems-level paradox: experience that should reduce rediscovery can instead enlarge the inference working set. We investigate whether persistent agent knowledge can remain broad while model-active context is compiled narrowly for each mission. We first measure historical-context amplification across three agent runtimes. In two controlled OpenAI Codex experiments, resuming a historical session required approximately $6.70\text{--}6.73\times$ the model input of a fresh sufficient-context execution while preserving identical 10/10 outcomes. In a single Google Gemini CLI replication pair, deliberately unrelated synthetic history produced a $45.57\times$ input amplification while both arms scored 10/10. In one Anthropic Claude Code budget-feasibility test, a large historical session consumed approximately 669.7K input/cache components and exhausted a configured budget before completing, whereas a fresh session completed the same self-contained task at 10/10 using 22.3K input/cache components; because the historical arm did not complete, we do not treat this as an outcome-equivalent ratio.

Motivated by these observations, we propose the *Agentic Context Lifecycle Architecture* (ACLA) and a deterministic *Context Compiler* that separates persistent memory, authority resolution, active context, caching, and validation. The design was developed through a staged R2A–R2D program: executable contract and invariants (R2A), standalone deterministic compiler (R2B), read-only shadow evaluation against a real enterprise engineering agent (R2C), and controlled model execution (R2D). In the enterprise shadow, the compiler reduced an estimated 16.4K-token live analysis context to 9.4K tokens (42.66% estimated structural reduction), preserving 100% of mandatory facts and authority coverage while injecting zero explicitly superseded facts; this R2C result is not provider-metered input. In two replicated R2D A/B executions using the same model, reasoning effort, output contract, read-only constraints, and zero tools, the compiled working-set treatment preserved a perfect 12/12 deterministic outcome in all arms while reducing provider-reported model input by 28.42% and 24.99%, respectively. The treatment arm was not faster, and cache behavior differed materially across replicates, reinforcing that model-input reduction, caching, latency, compute, and monetary cost are distinct dimensions.

A post-review evidence layer then tested three boundaries raised during external scientific review. A three-arm structural experiment separated serialization from admission: identical semantics shrank from 4,650 to 3,417 UTF-8 bytes under compact serialization (26.5161%), and deterministic admission further reduced the same compact representation to 2,701 bytes (20.9541% relative to the compact all-semantics arm; 41.9140% end-to-end structural difference). These structural measurements do not decompose the historical provider-metered R2D effect. A held-out cold-chain logistics suite with a contract-derived validator frozen before compiler execution passed 28 of 29 deterministic predicates; the sole preserved mismatch concerned the reported exclusion stage for an equivalent record (`DUPLICATE` expected versus `AUTHORITY_OVERRIDDEN` observed), and the suite was not repaired or rerun. Finally, falsification showed that a wrong highest-ranked live observation can propagate through deterministic precedence. These results narrow the mechanism framing, causal interpretation, and correctness boundary without rewriting the frozen R2A–R2D evidence.

We therefore make a bounded claim: for the enterprise mission studied, authority-aware context compilation reduced actual model input without reducing validated task quality. The contribution is not a new transformer architecture; it is an agent-runtime systems architecture that treats model context as a scarce working set rather than a persistence layer. We position the work relative to memory hierarchies, retrieval-augmented generation, prompt compression, long-context evaluation, agent memory, context engineering, and contemporaneous 2026 work on context lifecycles and working sets. We conclude with a measurement framework that directly measures Working-Set Amplification, Historical Context Tax, and Fresh Context Reduction while reserving outcome-normalized economics for future instrumentation, and with a falsifiable design principle: *the future agent should remember far more than it processes*.

Keywords: agentic AI; context engineering; memory systems; working set; long-horizon agents; retrieval; prompt compression; context caching; enterprise agents; AI systems architecture

1 Introduction

AI agents are increasingly designed to operate across hours, days, or recurring sessions rather than as isolated prompt-response systems. During those lifetimes they accumulate state: dialogue, system instructions, tool definitions, tool results, plans, summaries, retrieved documents, execution evidence, failed hypotheses, user preferences, checkpoints, and model-generated notes. This state is useful because an agent that has already learned a system should not rediscover it from first principles on every turn. Yet the mechanisms used to preserve experience and the mechanisms used to supply inference context are frequently coupled. When they are coupled, an agent can become more expensive to operate precisely because it has become more informed.

The tension is not the same as the familiar problem of finite context-window capacity. Modern language models can accept increasingly long inputs, and research has extended effective sequence memory through recurrence, compressed memories, and retrieval (Y. Chen et al., 2025; Dai et al., 2019; Lewis et al., 2020; Rae et al., 2020). The question studied here is different: *even when a model can accept the history, should the agent make that history computationally active for the current mission?*

This distinction becomes important in long-running agents because the context window serves at least two roles that should not be conflated. It is an inference substrate—the information presently available to the model—but it is often also used as a de facto persistence mechanism for prior interaction. The former should be bounded by task requirements; the latter may grow with experience. Classical systems architecture offers a useful analogy. Denning’s working-set model separated the pages a program was actively using from the larger address space it could potentially access (Denning, 1968). The analogy is imperfect—language-model context is not virtual memory—but it motivates a precise systems question: can an agent maintain a broad durable knowledge base while compiling a much smaller, mission-specific inference working set?

In the present prototype, “mission working set” has a narrower meaning than a complete operating-system demand-paging mechanism. The Context Compiler implements authority-aware admission control and a *a priori* mission working-set estimation; it does not implement a demand-fault path that pages omitted context back into an active execution. The operating-systems analogy therefore motivates bounded activation, but the v1 mechanism should not be read as Denning-complete virtual memory.

We answer that question through a staged empirical program. First, we establish that historical session state can impose a measurable *Historical Context Tax* even when the immediate task is self-contained. We observe this effect in controlled tests across Codex, Gemini CLI, and Claude Code. These tests are not intended as a vendor ranking. The magnitude varies dramatically by runtime, cache state, session representation, model, and

serving system. The common observation is simply that accumulated session state can remain computationally active even when a fresh sufficient context yields the same validated outcome.

Second, we propose the *Agentic Context Lifecycle Architecture* (ACLA), whose central runtime construct is a deterministic *Context Compiler*. The compiler does not merely retrieve semantically similar text. It resolves current authority, temporal validity, provenance, mission scope, mandatory constraints, conflicts, and context budget before producing a model-active working set. Importantly, it emits a separate audit/provenance plane so that explainability and reproducibility do not themselves become inference bloat.

Third, we test the architecture progressively rather than jumping directly to an opaque memory platform. R2A freezes an executable contract and invariants. R2B implements a deterministic reference compiler and validates it on a synthetic stress fixture. R2C binds the compiler read-only to a real enterprise engineering agent, where current runtime evidence can contradict historically validated knowledge. R2D then runs a controlled model A/B experiment: a raw real context packet versus the R2C-compiled working set, with the same mission, model, reasoning effort, output schema, validator, safety constraints, and fresh execution per arm.

The primary empirical result is modest but consequential. Across two replicated R2D pairs, the compiled working-set treatment reduced provider-reported model input by 28.42% and 24.99%, respectively, while all four executions achieved the full 12/12 deterministic quality score and made zero tool calls. These are two observed treatment effects, not a population estimate. They do not imply equivalent monetary-cost, compute, or latency reduction. In fact, the compiled arms were slightly slower. The result establishes only that, for the bounded enterprise mission studied, the smaller authority-aware working-set treatment preserved the validated outcome.

Research question. We ask:

Can an agent persist broad historical knowledge while compiling a materially smaller mission-specific working set without reducing validated task quality?

Hypotheses. Our primary hypothesis is:

H1. For bounded agentic missions, separating persistent memory from model-active context and applying deterministic authority-aware context compilation can reduce model input while preserving validated outcome quality.

H0. Reducing the broader context materially degrades validated performance because information removed by compilation is necessary to complete the task correctly.

Contributions. We make six contributions:

1. controlled cross-runtime evidence that historical session state can materially amplify model input even when current-task information is sufficient;
2. an explicit separation among persistent memory, audit/provenance evidence, and model-active context;
3. an experimentally instantiated *authority- and temporal-aware* context-compilation architecture with deterministic precedence rules, invariants, and a context budget;
4. a staged R2A–R2D evaluation methodology from executable contract to deterministic compiler, real enterprise shadow, and replicated model A/B;
5. a measurement vocabulary and initial instrumentation that directly measures Working-Set Amplification (WSA), Historical Context Tax (HCT), and Fresh Context Reduction (FCR), structurally represents Context Provenance Coverage (CPC), Stale Context Rate (SCR), and Tool Disclosure Ratio (TDR), and explicitly labels CE, CU, CCR, CRE, CPVO, and CES as proposed or future deployment metrics;
6. a bounded empirical validation in which two controlled R2D replicate pairs reduced provider-reported model input by 28.42% and 24.99%, respectively, while preserving the full deterministic outcome contract.

We deliberately do *not* claim that context lifecycle management is a new field, that the term “working set” is novel in LLM systems, or that the compiler metaphor for model context originates here. In particular, Letort’s April 2026 Context Compilation Theory already formalizes context as selected, transformed, governed, optimized, and lowered through a Context IR under constrained optimization (Letort, 2026); SkillRAE uses context compilation for retrieved skills (Meng et al., 2026); Latent Context Compilation compiles long context into portable latent buffer tokens (Zaju Li et al., 2026); and Agent Context Compilation recompiles successful trajectories into long-context training examples (Su et al., 2026). MemGPT draws on operating-system memory hierarchies (Packer et al., 2023); prompt-compression systems optimize input density (Jiang, Q. Wu, C.-Y. Lin, et al., 2023; Jiang, Q. Wu, Luo, et al., 2024); and recent work develops demand paging, pruning, and agentic context lifecycle abstractions (Dadhich, 2026; X. Li et al., 2026; Lodha et al., 2026; Mason, 2026). Our distinctive claim is narrower: as of 28 August 2026, we are not aware of a single prior system in the reviewed literature that combines *deterministic heterogeneous-source authority precedence, temporal supersession, separate audit and model-wire planes, cross-runtime historical-amplification measurement, and a staged R2A–R2D validation ending in replicated raw-versus-compiled provider-input measurement on a real enterprise state problem*. This is a bounded literature-audit statement, not a priority claim over context compilation as a field.

Table 1: Core distinctions used in this work.

Distinction	Interpretation
Memory $\neq$ context	Persistence may be broad; inference context is mission bounded.
Session identity $\neq$ working-set identity	Resuming a session need not imply replaying its active history.
Retrieval $\neq$ authority	Similarity proposes candidates; it does not determine present truth.
Capacity $\neq$ requirement	A model’s maximum context is not a target utilization level.
Caching $\neq$ context selection	Cache reuse changes compute economics for selected tokens; it does not decide whether those tokens should be selected.
Audit evidence $\neq$ model evidence	Provenance and exclusion records must remain recoverable without automatically entering model input.
Completion $\neq$ success	A durable success boundary requires validation, not merely a generated answer.

2 Problem Definition and System Boundaries

2.1 The persistence–computation conflation

Let M_t denote the agent’s durable memory at time t , and let C_t denote the tokens supplied to the model for a particular inference turn. A naive long-horizon architecture tends toward

$$C_t \approx C_{t-1} \cup \Delta_t, \quad (1)$$

where prior context is carried forward and new observations are appended or compacted. Even with periodic summarization, this makes session continuity the organizing unit for inference. We instead distinguish durable state from the current mission working set:

$$W_t = \Gamma(M_{\leq t}, E_t, A_t, Q_t, B_t), \quad (2)$$

where E_t is current evidence, A_t is authority and temporal state, Q_t is the mission specification, B_t is a context budget, and Γ is a compilation function. The model receives W_t , not the full historical store.

This difference yields a design asymmetry:

$$\begin{aligned} |M_t| &\text{ may grow with experience,} \\ |W_t| &\text{ should track mission need.} \end{aligned} \quad (3)$$

The goal is not minimality for its own sake. Context that is mandatory for correctness or safety must not be removed merely to save tokens. The goal is the *smallest sufficient validated working set*, not the shortest prompt.

2.2 Why this is an agent-runtime problem, not a transformer claim

The transformer architecture establishes a mechanism for token-to-token interaction (Vaswani et al., 2017); long-context research modifies or extends how those interac-

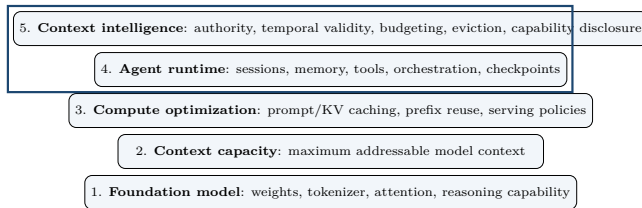


Figure 1: Five-layer model. This work primarily contributes at the agent-runtime and context-intelligence layers (4–5), while measuring consequences at the inference boundary.

tions scale (Dai et al., 2019; Rae et al., 2020). Our intervention does not alter model weights, attention kernels, position encodings, or training. It acts before inference, at the agent-runtime layer that decides what state becomes model input. Accordingly, the observed system behavior is a joint function of at least five layers:

This layered view prevents an important category error. A large context window can increase *capacity* without improving *context efficiency*; prompt caching can reduce the cost of repeated prefixes without reducing the size of the logical working set; and durable memory can improve continuity while still increasing active processing if the runtime re-injects too much history. The architecture must therefore manage these concerns separately.

3 Related Work and Novelty Boundary

3.1 Long-context sequence models and memory hierarchies

Transformer-XL introduced segment-level recurrence to model dependencies beyond a fixed-length segment (Dai et al., 2019). Compressive Transformers further compressed older memories to extend effective history (Rae et al., 2020). MELODI uses hierarchical compression across layers and context windows, separating short-term and longer-term memory representations while reducing memory footprint (Y. Chen et al., 2025). These works demonstrate that not all historical information must remain represented at full fidelity inside the immediate computation graph.

The operating-systems analogy has also appeared directly in LLM systems. MemGPT treats context management as virtual memory, moving information between context and external storage (Packer et al., 2023). vLLM’s PagedAttention applies paging ideas to KV-cache memory allocation on the serving side (Kwon et al., 2023). Most directly, Mason proposes demand paging for LLM context windows and explicitly frames the context window as a scarce cache-like resource rather than the complete memory system (Mason, 2026). AgentRM likewise introduces a three-tier Context Lifecycle Manager with adaptive compaction and hibernation inside an OS-inspired agent resource manager (She, 2026). These works reinforce that memory tiering, working sets, paging, and lifecycle management are prior art. Our work is aligned with this

direction but differs in the unit of policy: we compile a mission working set using explicit heterogeneous-source authority, temporal validity, provenance, and safety constraints, and we preserve a separate audit representation.

3.2 Retrieval, compression, and long-context quality

RAG combines parametric and retrieved non-parametric memory (Lewis et al., 2020). Prompt-compression methods such as LLMingua and LongLLMLingua explicitly reduce input tokens under a semantic-fidelity objective (Jiang, Q. Wu, C.-Y. Lin, et al., 2023; Jiang, Q. Wu, Luo, et al., 2024). ReadAgent creates gist memories of long text and retrieves original passages when needed (Lee et al., 2024). Recent coding-agent systems make the selection problem more task-specific: SWE-Pruner learns goal-conditioned line retention, LaMR separates semantic evidence from dependency support, TACO evolves observation-compression rules, and Paritok-4B performs intent-conditioned extractive compression (Ren et al., 2026; Shi and L. Chen, 2026; J. Wang et al., 2026; Y. Wang et al., 2026). These techniques demonstrate that selective or compressed context can preserve or improve downstream performance, while also showing that context reduction itself is not novel to this work.

However, relevance is not equivalent to authority. In enterprise systems, an older configuration fact can be semantically perfect yet operationally wrong because a newer runtime observation supersedes it. Likewise, a previously validated architecture can remain historically true while a current dependency is unavailable. Our Context Compiler therefore uses retrieval as candidate generation, then applies an authority and temporal-resolution stage before selection. We summarize the principle as: *RAG proposes; the Context Compiler disposes*.

Long-context benchmarks also show why capacity alone is insufficient. Liu et al. (2024) show that model performance can depend strongly on where relevant information appears in long input. LongBench and RULER expose performance degradation as context length and task complexity increase (Bai et al., 2024; Hsieh et al., 2024). These findings motivate treating context as a quality-sensitive resource rather than free storage.

3.3 Agent memory and long-horizon behavior

Language-agent architectures increasingly separate forms of memory and action. ReAct interleaves reasoning and action (Yao et al., 2023); Cognitive Architectures for Language Agents provides a broader framework connecting working memory, episodic memory, semantic memory, and procedural components (Sumers et al., 2024). Reflexion stores verbal feedback across trials (Shinn et al., 2023); Generative Agents retrieve, reflect on, and plan over memory streams (Park et al., 2023); Agent Workflow Memory selectively reuses induced workflows (Z. Z. Wang et al., 2025); A-MEM and MemOS develop structured, evolving

memory systems (Zhiyu Li et al., 2025; Xu et al., 2025). LongMemEval operationalizes long-term memory quality through extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention (D. Wu et al., 2025).

Our focus is complementary. We do not propose a new memory store. We ask what happens *after* memory exists: which portion should become computationally active for a specific mission, under whose authority, for how long, with what provenance, and at what token budget.

3.4 Context engineering and contemporaneous convergence

Industry practice is converging on similar concerns. Anthropic describes context as a finite resource and recommends finding a minimal high-signal token set, dynamically retrieving information, compacting histories, and limiting tool surfaces (Anthropic Applied AI Team, 2025). OpenAI prompt caching and Google Gemini context caching optimize repeated prefixes, but both remain compute/cost optimizations on content that has already been selected (Google, 2026a; OpenAI, 2024). Google’s long-context guidance explicitly notes that unnecessary tokens should generally not be passed and that longer queries typically increase latency (Google, 2026b). OpenAI’s memory work emphasizes freshness, continuity, relevance, and scalable synthesis (OpenAI, 2026a). OpenAI’s current model guidance also exposes an explicit reasoning-context choice between `current_turn` and `all_turns`, recommending `current_turn` when earlier reasoning is no longer relevant (OpenAI, 2026b).

The 2026 literature makes novelty discipline especially important. As of 28 August 2026, the following closely related 2026 works are contemporaneous arXiv preprints rather than peer-reviewed archival publications; we use them to delimit novelty, not as settled empirical authority. Dadhich independently defines Agentic Context Management as a lifecycle problem spanning ingestion, scoping, anticipation, compaction, and consolidation (Dadhich, 2026). Khan and Lipizzi study memory inside the agent loop and show a trade-off between retrieval latency and restating a working set in tokens (Khan and Lipizzi, 2026). Chang et al. characterize agentic serving and identify a control-plane tax from auxiliary calls, tool schemas, and observations (Chang et al., 2026). Mohammadi et al. use “working set” to study fact availability and coherence debt in repository-scale coding agents, reporting that harnesses with identical task success can differ by more than an order of magnitude in token consumption (Mohammadi et al., 2026). Mason’s demand-paging work and AgentRM’s three-tier Context Lifecycle Manager are likewise contemporaneous arXiv preprints (Mason, 2026; She, 2026).

Our claim is therefore not priority over “context lifecycle” or “working set” as concepts. The distinctive combination in this work is: (i) controlled cross-runtime measurement of historical-session amplification, (ii) explicit

authority and temporal validity distinct from semantic relevance, (iii) deterministic compilation with auditable inclusion/exclusion reasons, (iv) separation of audit and model-active wire planes, and (v) a staged R2A–R2D program culminating in replicated provider-reported model-input reduction at unchanged validated quality.

3.5 Novelty boundary after targeted 2026 audit

A targeted literature audit conducted through 28 August 2026 materially narrows the novelty boundary. Several contemporaneous systems overlap with individual mechanisms in ACLA. Letort’s Context Compilation Theory is the closest conceptual predecessor to the generic compiler framing: it defines a Context IR, constrained compilation objective, governance, lowering, and compilation-oriented evaluation (Letort, 2026). Scroll separates a persistent append-only event log from a bounded model-visible “working view,” retaining provenance and recoverability while letting model-written programs decide what is printed into context (Y. Lin et al., 2026). MemIR separates raw evidence, retrieval cues, and truth-bearing claims and restricts factual authorization to supported claim atoms (Jin et al., 2026). Supersede isolates temporal fact-currency as a distinct long-term-memory failure mode (Patel, 2026). Less Context, Better Agents demonstrates on an enterprise MCP workflow that pruning and summarization can outperform full-history retention while using substantially fewer tokens (Lodha et al., 2026). ACM gives agents lossless context-editing tools and external memory (X. Li et al., 2026); Mason provides demand paging and working-set eviction (Mason, 2026); AgentSysBench quantifies tool-schema and observation overhead as an LLM control-plane tax (Chang et al., 2026); and repository-scale work shows that stale conventions can be worse than missing context (Mohammadi et al., 2026).

These overlaps rule out broad novelty claims such as “agents need bounded working sets,” “context should be compiled,” “stale facts should be removed,” “full history is inefficient,” or “task-conditioned pruning reduces token use.” The remaining research contribution is the *systems composition and validation*: ACLA makes authority precedence over heterogeneous evidence classes an explicit deterministic stage; treats temporal state as orthogonal to semantic relevance; preserves an audit/provenance plane while lowering a separate model-wire representation; freezes these semantics as executable invariants before implementation; and evaluates them through synthetic stress, real read-only enterprise shadowing, and replicated provider-metered A/B execution. This is a composition-level contribution: each constituent idea has close prior art, while our claim concerns their explicit integration and staged empirical validation. We therefore frame the paper as an *authority-aware empirical instantiation and validation of context-lifecycle architecture*, rather than as the origin of context compilation itself.

Table 2: Conservative positioning against representative related approaches. The table records each work’s stated primary mechanism and evaluation setting rather than assigning broad capability checkmarks. The 2026 entries marked “preprint” were not peer-reviewed archival publications as of 28 August 2026.

Work / approach	Primary mechanism	Active-context intervention	Authority / temporal treatment	Evaluation basis
MemGPT (Packer et al., 2023)	virtual context / memory tiers	moves data across tiers	not a primary mechanism	document analysis; multi-session chat
LLM4Intelligence family (Jiang, Q. Wu, C.-Y. Lin, et al., 2023); Jiang, Q. Wu, Luo, et al., 2024)	prompt compression	token-level / passage compression	not a primary mechanism	controlled downstream task evaluation
ReadAgent (Lee et al., 2024)	git memory + selective lookup	git compression + passage retrieval	not a primary mechanism	long-document QA benchmarks
Mason 2026 (Mason, 2026)	demand paging	eviction, faults, pinning	not a primary mechanism	production/replay measurements; preprint
AgentRM 2026 (She, 2026)	OS-inspired resource manager + context lifecycle tiers	adaptive compaction + liberation	not an explicit heterogeneous-source precedence mechanism	framework analysis + system evaluation; preprint
Dadliah 2026 (Dadliah, 2026)	context-management lifecycle	scoped retrieval + validated compaction	scope/provenance emphasized; no explicit precedence hierarchy	benchmark reference implementation; preprint
Mohammadi et al., 2026 (Mohammadi et al., 2026)	fact availability / coherence debt	supplies/withholds required facts	state convention conflicts measured	controlled coding-agent workloads; preprint
Letort 2026 (Letort, 2026)	Context IR + compilation theory	governed selection / transformation / lowering	governance + provenance intrinsic to compilation	theory + worked artifacts; Zenodo preprint
Musallr 2026 (Jin et al., 2026)	typed memory IR	claim-centered projection	factual authorization + provenance; temporal grounding	LoCoLo + BEAM-100K; preprint
Scroll 2026 (Y. Lin et al., 2026)	persistent Session Environment	bounded working view + recoverable eviction	typed provenance; temporal queries via programmatic access	LongMemEval / BEAM / LOCA; preprint
Less Context 2026 (Lodha et al., 2026)	pruning + summarization	recent-tool retention / compact summary	stable-state mitigation, not explicit authority hierarchy	enterprise MCP workflow; preprint
This work	authority-aware context compilation	deterministic admission under budget + separate model wire	explicit heterogeneous-source authority + temporal precedence	R2C enterprise shadow + replicated R2D A/B

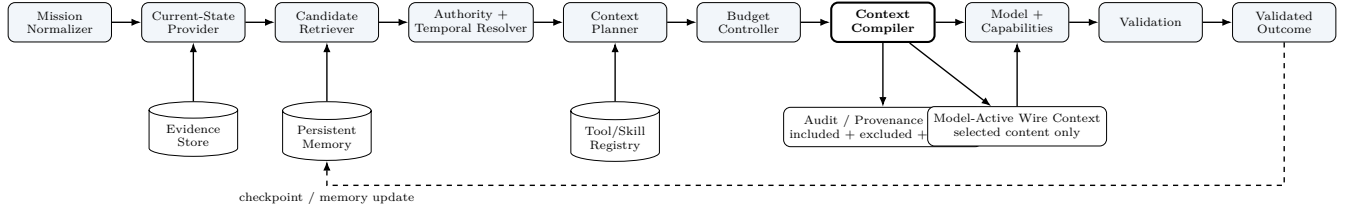


Figure 2: Agentic Context Lifecycle Architecture (ACLA). Persistent stores can grow; the Context Compiler emits a bounded model-active working set and a separate audit/provenance artifact.

4 Agentic Context Lifecycle Architecture

4.1 Design objective

ACLA treats persistent memory, inference context, caching, and validation as different lifecycle concerns. Conversation history becomes an evidence source, not the default execution substrate. The runtime’s job is to transform a mission and a universe of possible information into the smallest sufficient working set that remains valid under current authority.

4.2 Authority and temporal validity

Memory is not truth. Retrieval is not authority. The compiler therefore attaches an authority class and temporal state to each candidate. The R2A reference implementation uses the following default precedence, which is intentionally configurable by domain:

1. live enterprise authority;
2. verified runtime result;
3. validated artifact;
4. approved architecture decision;
5. verified memory;
6. current user assertion;
7. historical tool observation;
8. model-generated memory;
9. unverified historical text.

This ordering is not a universal epistemology. It is an explicit policy that prevents silent merging of observations with different authority. A live “service unavailable” result can therefore outrank a historical artifact showing that the service was previously proven, without erasing the historical fact.

4.3 Candidate scoring and feasibility constraints

For candidate context item c_i , define normalized attributes:

$$R_i \in [0, 1] \quad \text{mission relevance,} \quad (4)$$

$$A_i \in [0, 1] \quad \text{authority,} \quad (5)$$

$$T_i \in [0, 1] \quad \text{temporal validity,} \quad (6)$$

$$V_i \in [0, 1] \quad \text{validation confidence,} \quad (7)$$

$$U_i \in [0, 1] \quad \text{mission utility,} \quad (8)$$

$$K_i > 0 \quad \text{estimated token cost.} \quad (9)$$

The R2A reference design uses a multiplicative heuristic

$$CV_i = R_i A_i T_i V_i U_i, \quad (10)$$

with context value density

$$D_i = \frac{CV_i}{\max(K_i, 1)}. \quad (11)$$

These values are *not probabilities*. They are deterministic ranking signals after hard policy checks. Let $\mathcal{C}$ be the candidate universe, $\mathcal{M}$ mandatory items, $\mathcal{S}$ explicitly superseded items, and B the token budget. A feasible working set W must satisfy at least

$$\mathcal{M} \subseteq W, \quad (12)$$

$$W \cap \mathcal{S} = \emptyset \quad \text{by default,} \quad (13)$$

$$\sum_{c_i \in W} K_i \leq B, \quad (14)$$

$$\text{Prov}(c_i) \neq \emptyset \quad \forall c_i \in W. \quad (15)$$

Superseded items may be admitted only when the mission explicitly requests historical analysis.

The compiler then selects high-value feasible candidates under budget. Hard invariants dominate score: a mandatory low-density fact cannot be dropped, an unresolved conflict cannot be silently collapsed, and an unknown field cannot be inferred simply to improve completeness.

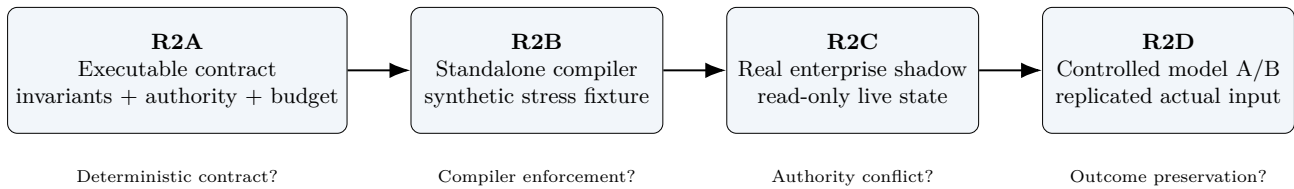


Figure 3: R2A–R2D progression. Each stage answers a different falsifiable question and freezes its result before the next stage.

4.4 R2A invariants

R2A froze seventeen invariants before implementation. They form the safety contract against which later stages are evaluated:

- I1. Memory $\neq$ context.
- I2. Session history $\neq$ required working set.
- I3. Retrieval $\neq$ authority.
- I4. Current authority outranks historical memory for present state.
- I5. Superseded facts are excluded by default.
- I6. Mandatory context cannot be dropped for token savings.
- I7. Tool disclosure is context.
- I8. Skill disclosure is context.
- I9. Cache optimization occurs after context selection.
- I10. Raw evidence is preserved.
- I11. Context eviction $\neq$ data deletion.
- I12. Every included fact has provenance.
- I13. Every excluded candidate has a reason.
- I14. Unknown $\neq$ inferred.
- I15. Conflict $\neq$ silently merged.
- I16. Validation defines durable success.
- I17. Shadow mode performs zero execution-side writes.

These invariants remain the frozen R2A contract. Post-review R2E-C does not rewrite them; it narrows the interpretation of I4 by showing that precedence assumes the strongest authority is itself sufficiently reliable.

R2D added a further implementation insight without changing selection policy: **auditability must not automatically become model input**. The compiler can retain rich IDs, scores, provenance, exclusion reasons, hashes, and budgeting metadata while serializing only the admitted semantic content plus authority/validation semantics into the model wire format.

5 R2A–R2D Research Program

The architecture was evaluated as a sequence of increasingly realistic gates. This staging was designed to prevent premature integration and to make failures interpretable.

5.1 R2A: executable contract

R2A defined the mission schema, candidate schema, authority precedence, temporal states, conflict representation, context budget, capability descriptors, output contract, telemetry fields, and shadow-mode constraints. The key design decision was to make the first compiler deter-

ministic. A learned or LLM-based planner could later improve candidate generation, but introducing another model to decide which tokens a model should see would confound the first evaluation and add a new failure/cost surface.

5.2 R2B: deterministic reference compiler

R2B implemented the contract as a standalone compiler with no model calls, embeddings, vector database, backend calls, or repository writes. A synthetic HELIOS fixture combined ten authoritative mission facts with 3,500 unrelated NEBULA records. The candidate pool was estimated at 234,740 tokens; the compiler selected a 240-token context, avoiding approximately 99.90% of candidate tokens while preserving all ten authoritative facts. This result is intentionally treated as an implementation stress test, not evidence of real-world compression. A deterministic filter should perform extremely well when irrelevant history is deliberately separable.

5.3 R2C: real enterprise read-only shadow

R2C bound the compiler to a real enterprise engineering-agent runtime through read-only HTTP observations. No model, backend, or repository write was permitted. The public release does not expose the private runtime snapshot; instead it preserves cryptographic identifiers for the frozen evidence and provides a synthetic authority-conflict fixture that preserves the scientific relationship under test. The fixture is not represented as the original enterprise dataset, and the exact R2C/R2D enterprise token counts are therefore not independently replayable from public data. This case is materially harder than HELIOS because the context contains semantically related but temporally different facts: validated architecture decisions, historical failures, known-good baselines, negative knowledge, runtime status, governance constraints, and capabilities.

The live analysis state contained 16 known-good baselines, 23 negative-knowledge records, eight locked architecture decisions, and an available tool inventory of 25, of which three were identified as relevant. The critical test involved authority conflict. Historical evidence established that a backend execution path and bridge had previously been proven, but current live runtime status simultaneously reported `enterprise-backend_CONNECTION_FAILED`, `EXECUTION_BRIDGE_UNAVAILABLE`, and `bridgeReachable=false`. A relevance-only system

could retrieve both and leave resolution to the model. R2C encoded both facts while ensuring that current runtime evidence determined *present availability*.

R2C compiled an estimated 16,404-token live analysis context to 9,406 tokens, an estimated 42.66% reduction. It preserved 6/6 mandatory facts, achieved authority coverage 1.0 and provenance coverage 1.0, and injected zero explicitly superseded facts. All safety counters remained zero. The baseline token count was a deterministic estimate from serialized live JSON, not provider-metered model input, so it is not used as the decisive performance claim and must not be interpreted as cost, compute, or latency reduction.

5.4 R2D: controlled model A/B

R2D tested whether the structurally smaller context remained sufficient for actual model execution. The intervention was the complete context-treatment bundle rather than semantic selection alone:

Control A: fresh model execution with the raw enterprise engineering-agent GET-only evidence packet.

Treatment B: fresh model execution with the R2C-admitted working set lowered into the compact model-wire representation.

Accordingly, R2D supports a causal comparison of the *raw-context treatment versus the compiled-working-set treatment*; it does not separately identify the contribution of admission, ordering, serialization, or provider/runtime overhead.

Both arms used the same model (gpt-5.5), medium reasoning effort, mission, output schema, deterministic validator, empty read-only workspace, and zero tools. Each arm was a fresh execution rather than a resumed session. Replicate 1 ran A then B; replicate 2 reversed the order (B then A) to reduce simple order effects. The expected-answer contract, output schema, selection decisions, validator logic, and 20% engineering acceptance threshold were frozen before valid model execution and remained identical across A and B. A run passed only if it scored 12/12 on current blocker, current runtime reason, application readiness, bridge reachability, enterprise-backend write authority, recorded next action, current safety of that action, authority class for current availability, and two prohibited reinvestigation scopes. Any tool-like item failed the safety gate. Because the validator was authored within the same research program and encodes the mission’s authority semantics, it should not be interpreted as an external benchmark; identical 12/12 control and treatment outcomes reduce simple treatment-favoring circularity but do not remove this limitation.

Two harness defects were discovered before the valid R2D run. First, a Windows command-line limit required moving the prompt from argv to standard input. Second, the initial treatment serialized the compiler’s full audit artifact rather than only the active semantic plane, making the treatment larger than intended. A subsequent structured-output schema used a keyword unsup-

ported by the provider and was rejected before inference. These failed harness attempts are retained as evidence but excluded from scientific results. The final HF02 run separated the model wire format from audit metadata, statically validated the schema, and preserved the frozen selection decisions and 20% threshold.

5.5 Post-review R2E checks

External scientific review identified three experimental boundaries that were not resolved by the frozen R2A–R2D program. We therefore added a separate post-review evidence layer without modifying the historical measurements.

R2E-A: selection versus serialization. R2D compared a raw evidence packet with a compiled compact model-wire treatment, so its provider-input difference cannot be attributed to semantic admission alone. R2E-A separated the structural components with three frozen arms on the public synthetic mission. Arm A retained all semantic content in verbose JSON (4,650 UTF-8 bytes). Arm B retained the same semantics in compact JSON (3,417 bytes), a 26.5161% serialization reduction. Arm C used the compiler-selected content in the same compact representation (2,701 bytes), a further 20.9541% reduction relative to B and a 41.9140% structural difference relative to A. These are exact application-payload measurements, not provider-metered input tokens, and they are not used to retroactively decompose the two historical R2D provider-input reductions.

R2E-B: held-out frozen-validator check. To reduce validator/compiler co-development risk, we froze a four-case held-out suite in an unrelated cold-chain medical-logistics domain before executing the unchanged v1.0.0 compiler. The validator was contract-derived, not independently human-authored and not model-authored. It contained 29 deterministic predicates across successful admission, mandatory cross-scope blocking, equal-authority conflict blocking, and required-authority absence. Twenty-eight predicates passed. The sole preserved mismatch expected an equivalent SEAL-DUP record to be excluded as DUPLICATE; the compiler excluded the same record earlier as AUTHORITY_OVERRIDDEN. We did not change the expected outcome or rerun the suite to obtain a perfect score. The result therefore supports broad contract conformance while exposing an observable pipeline-stage semantics mismatch; it is not evidence of independent human validation or population-level generalization.

R2E-C: false-live falsification of authority precedence. A falsification case supplied a wrong current live observation at the strongest authority rank and contradictory, strongly validated weaker evidence. The frozen v1.0.0 compiler compiled the false current observation. A control with contradictory claims at equal authority

Table 3: Measured and structurally validated context-lifecycle metrics. “Measured” indicates numeric values were obtained in this study; “structural” indicates deterministic checks without token-normalized labels.

Metric	Definition	Interpretation	Status
Working-Set Amplification (WSA)	conditional ratio	Payload-WSA uses serialized application payload; Provider/Wire-WSA uses provider-reported model input. Both require a fixed mission/validator and comparison basis.	Measured
Historical Context Tax (HCT)	$I_{hist} - I_{fresh}$	Absolute input attributable to historical carry-forward in the historical-session experiments; raw-minus-compiled in R2D.	Measured
Fresh Context Reduction (FCR)	$1 - I_{fresh}/I_{hist}$	Fractional reduction from historical/raw input to sufficient/compiled input.	Measured
Stale Context Rate (SCR)	I_{state}/I_{inj}	Leakage of explicitly stale or invalid state. R2C observed zero superseded items injected, but did not token-normalize this rate.	Structural
Context Provenance Coverage (CPC)	N_{prov}/N_{claims}	Coverage of admitted claims with known provenance. R2C structural coverage was 1.0.	Structural
Tool Disclosure Ratio (TDR)	N_{shown}/N_{avail}	Capability-schema exposure. the platform-native relevance stage identified 3 of 25 tools (0.12); R2C received identities only for those three and selected 3/3 of the supplied set.	Qualified

correctly produced `BLOCKED_CONFLICT`. Thus deterministic authority precedence can relocate error from model inference to policy correctness, but it cannot guarantee factual correctness when the highest-ranked source is itself wrong. The post-review code line therefore adds a reliability-adjudication guard that blocks for re-observation/adjudication when a low-confidence strongest authority contradicts strongly validated weaker evidence; it does not silently allow weaker authority to win.

6 Measurement Framework

A central objective of this work is to move evaluation from “context window size” toward context *effectiveness*. Table 3 distinguishes metrics that were directly measured or structurally observed.

We use WSA as a *conditional family of ratios*, not as an intrinsic property of a context object. **Payload-WSA** is the serialized application payload divided by the smallest validator-sufficient payload under a fixed mission, validator, and representation. **Provider/Wire-WSA** is provider-reported model input divided by provider-reported input for the validator-sufficient comparison arm under a fixed provider/runtime configuration. Both are conditional on the mission, validator, representation, and runtime/provider. R2D reports Provider/Wire-WSA; R2E-A reports structural payload decomposition and is not substituted for provider telemetry. **Only WSA, HCT, and FCR are treated as directly measured context-economics metrics in this paper;** CPC, SCR, and TDR are structural or qualified diagnostics, while CE, CU, CCR, CRE, CPVO, and CES are reserved for future instrumentation.

The present study intentionally keeps these dimensions separate rather than manufacturing a single score. In particular, CE, CU, CCR, CRE, CPVO, and CES are definitions for subsequent instrumentation, not results claimed here. The metric names are operational definitions for this study rather than blanket priority claims: outcome-normalized cost formulations and stale-context

measures also appear in contemporary industry practice. Our contribution is to connect these measures to a single context-lifecycle validation chain and to distinguish measured, structural, qualified, and proposed metrics explicitly.

For R2D replicate j , we define

$$\text{WSA}_j^{\text{wire}} = \frac{I_{A,j}}{I_{B,j}}, \quad \text{FCR}_j = 1 - \frac{I_{B,j}}{I_{A,j}}, \quad (16)$$

and

$$\text{HCT}_j = I_{A,j} - I_{B,j}. \quad (17)$$

For R2D, the two measured Provider/Wire-WSA values are $1.3971\times$ and $1.3331\times$, and HCT is 8,873 and 7,443 input tokens (median 8,158). The corresponding FCR values are 28.4227% and 24.9866%. For cross-runtime historical experiments, the denominator is the fresh sufficient-context input. For Claude, because the historical run did not complete, we report an *input-component ratio* rather than calling it a quality-equivalent WSA.

7 Cross-Runtime Historical-Context Experiments

7.1 Method

The first experiment family asks whether resuming historical state can increase model input for a self-contained task whose required facts are all present in the immediate prompt. The task packet, called HELIOS, contains a small synthetic project state and asks for a structured answer. Tools are disabled. Fresh and historical executions use the same model and reasoning effort within each runtime where controllable. Quality is scored deterministically. The experiments are not identical across vendors because session and telemetry mechanisms differ; they are best viewed as evidence of a common phenomenon, not a single pooled benchmark. All experiments reported here were executed on 28 August 2026. The archived runtime identifiers include Claude Code CLI 2.1.227 and Gemini CLI

Table 4: Future deployment metrics. These definitions are not empirical contributions of the present R2D study and are retained only to make the proposed measurement agenda explicit.

Metric	Definition	Purpose	Status
Context Efficiency (CE)	I_{rel}/I_{inj}	Density of mission-relevant information; requires token- or item-level relevance labels.	Proposed
Context Utilization (CU)	I_{comp}/C_{window}	Separates available context capacity from intentional context use.	Proposed
Context Churn Rate (CCR)	I_{new}/I_{W_t}	Rate at which the active working set changes between turns.	Proposed
Context Reuse Efficiency (CRE)	$I_{rel, reuse}/I_{reuse}$	Distinguishes useful reuse from replay of irrelevant cached content.	Proposed
Cost per Validated Outcome (CPVO)	$Cost/N_{valid}$	Economic north-star that penalizes inexpensive but incorrect or unvalidated execution.	Proposed
Context Effectiveness Score (CES)	composite, not fixed	Potential future composite; deliberately not collapsed in this study because quality, staleness, cache, latency, and cost are separable dimensions.	Future

0.57.0. The final R2D run used Codex CLI 0.146.0; the exact Codex CLI build for the earlier historical-session pairs was not preserved in the compact evidence record and is therefore reported as unavailable rather than inferred.

7.2 Codex

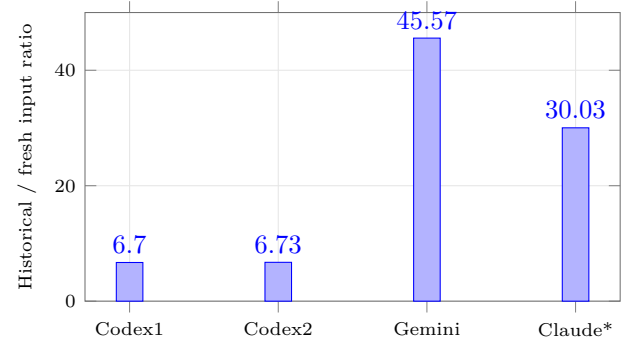
Two replicated Codex tests compared a resumed historical engineering session with a fresh session, both supplied the identical HELIOS packet, using the model identifier `gpt-5.5` as reported/configured by the Codex CLI, medium reasoning effort, no tools, and identical 10/10 outputs. In run 1, historical input was 95,273 tokens versus 14,227 fresh (WSA $6.70\times$, HCT 81,046, FCR 85.07%). In run 2, historical input was 95,813 versus 14,227 (WSA $6.73\times$, HCT 81,586, FCR 85.15%). The cache states differed sharply: one run reported only 1,408 cached input tokens in each arm, while the repeat reported 94,592 historical versus 13,696 fresh cached tokens. The amplification persisted.

7.3 Gemini CLI

The Gemini experiment seeded a session with 3,500 deliberately unrelated synthetic NEBULA records before supplying the same self-contained HELIOS task. Using `gemini-2.5-flash`, no tools, and the same structured validator, the historical run consumed 235,294 input tokens and the fresh run 5,163; both scored 10/10. This corresponds to WSA $45.57\times$, HCT 230,131, and FCR 97.81%. The historical run took 93.3 seconds versus 7.45 seconds fresh. No context-cache tokens were reported on the tested path. Because this result has one successful replication pair, it should be interpreted as strong evidence for that configuration rather than a population estimate.

7.4 Claude Code

The Claude Code experiment used a fork of a long historical session and the same self-contained HELIOS task under Sonnet 5, medium effort, a two-turn cap, and a configured maximum budget. The historical arm exhausted the configured budget before completing. It reported

**Figure 4:** Historical-input amplification. Claude* is an input/cache-component ratio whose historical arm did not complete; it is not an outcome-equivalent WSA.

approximately 669,704 input/cache-creation components and a CLI-reported estimated cost of about \$4.06; the fresh arm completed at 10/10 using 22,302 input/cache components and a CLI-reported estimated cost of about \$0.056. These CLI figures are estimates, not evidence of the amount ultimately billed. In this oversized request, the configured \$2 maximum-budget option also did not behave as a hard pre-execution ceiling on the CLI-reported estimate; we treat that as observed CLI behavior rather than a billing claim. The component ratio is approximately $30.03\times$ and fresh reduction 96.67%, but these are *not* quality-equivalent WSA/FCR because the historical arm failed. The result instead illustrates budget-feasibility and reliability pressure from an oversized active state.

The conclusion from Table 5 is intentionally narrow. There is no universal amplification multiplier. The magnitude depends on model, runtime, history encoding, system prompt, cache behavior, compaction, tokenizer, and serving layer. The common observation is that historical session state can remain computationally active even when current-task information is sufficient. This motivates architecture-level context selection independently of context capacity or caching.

Table 5: Controlled historical-context evidence. Ratios are runtime-specific and must not be averaged. Claude uses input/cache components and is not outcome-equivalent because the historical arm did not complete.

Runtime	Historical / raw input	Fresh input	WSA / ratio	FCR	Quality	Cache note
Codex R1	95,273	14,227	6.70×	85.07%	10/10 both	1,408 / 1,408
Codex R2	95,813	14,227	6.73×	85.15%	10/10 both	94,592 / 13,696
Gemini	235,294	5,163	45.57×	97.81%	10/10 both	0 reported
Claude Code	669,704 components	22,302 components	30.03× components	96.67%	historical failed; fresh 10/10	cache-create dominated historical

8 Enterprise Case Study: Authority-Aware Context Compilation

8.1 Why the enterprise case is harder

Synthetic distractors are useful for testing deterministic filtering, but enterprise memory is not composed of obviously irrelevant text. It contains facts that are all semantically related and may each have been correct at some point. Typical examples include a target system, deployment package, toolchain version, previously proven execution path, failed diagnostic hypothesis, approved architecture decision, and current runtime observation. The challenge is to select information by *mission and authority*, not merely similarity.

The enterprise case uses a research prototype that coordinates governed software-engineering workflows. For publication, private implementation names and the raw live snapshot are withheld. The paper reports only the scientific structure, aggregate counts, frozen cryptographic evidence identifiers, and public-safe labels. The experiment exposed no credentials, performed no backend or repository writes, and did not modify agent execution. The live context included current runtime status, an engineering checkpoint, validated known-good components, negative knowledge (facts or approaches known to be absent/false/prohibited), locked decisions, and relevant tool identities.

8.2 Historically true versus currently available

The most informative context conflict is shown in Figure 5. Historical evidence established that a backend execution path and bridge had been successfully used. Yet current runtime observations reported a connection failure and an unreachable bridge. These facts are not mutually exclusive:

$$\text{historically proven} \not\Rightarrow \text{currently available.} \quad (18)$$

A retriever optimized only for relevance could retrieve both statements. If both are presented without temporal/authority structure, the model must infer which controls present action. R2C makes the resolution explicit before inference: current runtime determines current availability; historical proof remains useful for deciding what *not* to redesign.

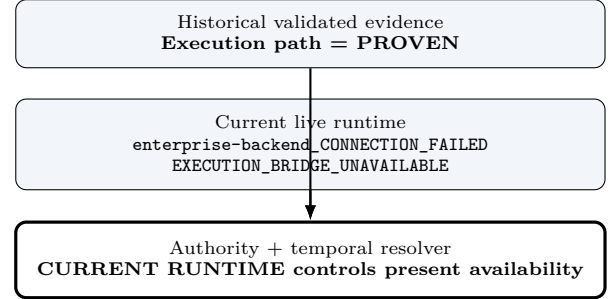


Figure 5: Enterprise authority example. The compiler preserves historical proof while preventing it from overriding present runtime availability.

8.3 R2C results

The R2C live snapshot was derived from eight GET-only endpoints, all returning HTTP 200. The analysis context reported 16 known-good baselines, 23 negative-knowledge items, eight locked decisions, 25 available tool schemas, and three tools selected as relevant. A separate factory context already reported its own optimization from 1,359 input tokens to 151 output-context tokens (88% compression), meaning the R2C baseline was *not* a deliberately naive “dump the whole conversation” architecture. This matters: the compiler was tested on top of an already context-aware enterprise agent.

R2C’s estimated baseline-to-compiled ratio was $1.744 \times$ (16,404 to 9,406 estimated tokens). Mandatory-fact preservation was 100%, authority coverage and provenance coverage were 1.0, and explicitly superseded-fact leakage was zero. Three named relevant tools were available to R2C and all three were selected; although the wider native agent reported 25 tools and four registered skills, identities for the other capabilities were not present in the collected evidence, so the compiler did not fabricate descriptors. This follows invariant I14: unknown $\neq$ inferred.

9 R2D: Controlled Model Evaluation

9.1 Experimental protocol

R2D converts the structural R2C result into a direct provider-measured inference test. The task asks the model to produce a current-state decision capsule that distinguishes the project checkpoint from current runtime availability. The validator checks twelve fields or required values. The critical semantic test is whether the model simultaneously preserves: (a) the checkpoint’s recorded

Table 6: R2C live shadow result. Token baseline is deterministic estimation, not provider-metered input.

Measure	Result
Estimated live baseline	16,404 tokens
Compiled working set	9,406 tokens
Estimated reduction	42.6603%
Mandatory facts preserved	6/6 (100%)
Authority coverage	1.0
Provenance coverage	1.0
Superseded facts injected	0
LLM / backend / repository writes	0 / 0 / 0

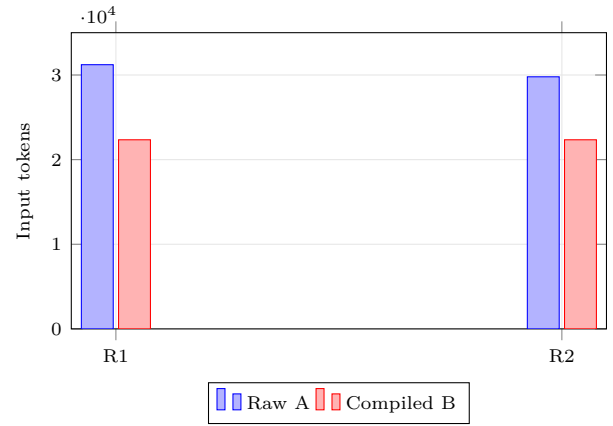
next action; (b) the fact that executing it is *not safe now* because current runtime readiness is blocked; and (c) the prohibition against redesigning historically proven backend/toolchain components.

The treatment context was derived solely from R2C-admitted items; no excluded candidate was reintroduced, and selection decisions were frozen before model execution. Shared mission instructions and the structured-output contract were supplied identically to both arms. The treatment used a compact model wire representation of the admitted content plus the authority/validation semantics required by the mission. The compiler’s audit envelope (candidate IDs, source IDs, token estimates, provenance manifest, exclusion manifest, context hash, and compiler metadata) was stored beside the prompt but not injected into the model. This distinction was discovered empirically during harness validation and is now a first-class architectural principle.

9.2 Results

Table 7 gives the provider-reported inputs. Every control and treatment arm scored 12/12. No tool-like items were emitted. Treatment input was lower in both replicates. The two observed reductions were 28.4227% and 24.9866%, exceeding the 20% engineering acceptance threshold frozen before execution by 8.4227 and 4.9866 percentage points, respectively. With two replicate pairs, these values are reported descriptively rather than as a population effect estimate.

The corresponding WSA values were $1.3971\times$ and $1.3331\times$; HCT was 8,873 and 7,443 tokens, respectively. Importantly, the deterministic pre-execution prompt estimator predicted approximately 44.19% payload reduction (18,631 versus 10,398 estimated tokens), whereas provider-reported model-input reductions were 28.42% and 24.99%. The experiment does not identify how much of this difference comes from shared system instructions, serialization, provider/runtime overhead, or other hidden components. Thus application-level context reduction should not be assumed to translate one-to-one into total model-input reduction.

**Figure 6:** R2D actual model input. Quality remained 12/12 in all four arms.

9.3 Caching and latency are separate dimensions

R2D provides useful negative evidence against simplistic economics claims. In replicate 2, the raw arm had 11,648 cached input tokens while the compiled arm had only 1,408, yet the compiled arm still had lower total model input. This supports a conceptual separation: caching optimizes processing of a selected working set; context intelligence changes the working set itself. Neither dominates universally.

Similarly, treatment B was slower in both replicates by approximately 8.0% and 9.4%. With only two pairs we do not infer a causal slowdown; provider scheduling, cache state, reasoning-token variation, and infrastructure noise can dominate wall-clock latency. The scientifically relevant conclusion is simpler: *fewer input tokens did not guarantee lower end-to-end latency in this experiment*. We therefore do not claim latency, compute, or monetary-cost reduction from the observed model-input reductions.

10 Discussion

10.1 The working-set problem

The experiments support a shift in architectural emphasis. Long-context capability answers “how much can the model address?” Context lifecycle management answers “what should the agent make active now?” These are different optimization problems. An agent that carries its entire experience forward may remain functionally correct while paying a repeated input-processing tax. Conversely, aggressive compression can improve efficiency but destroy a rare business-critical fact. The appropriate target is therefore not context minimization but *validated sufficiency*.

The R2D result is informative precisely because the task required multiple kinds of state: current runtime truth, historical validated proof, locked decisions, negative knowledge, and a next-action checkpoint. The treatment could not succeed merely by deleting history. It had to

Table 7: R2D controlled model A/B. A = raw real context packet; B = compiled model-active working set. All arms used gpt-5.5, medium reasoning effort, fresh sessions, identical validator/output schema, read-only empty workspace, and zero tools.

Replicate	Input tokens		B reduction	Quality		Cached input		Duration A/B
	A	B		A	B	A	B	
R1 (A→B)	31,218	22,345	28.4227%	12/12	12/12	1,408	1,408	15.74 / 17.00 s
R2 (B→A)	29,788	22,345	24.9866%	12/12	12/12	11,648	1,408	20.38 / 22.30 s
Observed range		24.9866–28.4227%		12/12 in all arms				

preserve the historically proven architecture as “do not redesign” knowledge while allowing current runtime evidence to block present execution. This is the operational meaning of authority-aware context.

10.2 Memory should grow differently from context

Persistent memory and active context should have different scaling laws. Durable memory can grow with experience because storage is comparatively cheap and evidence may be needed later for audit, recurrence, or a different mission. Active context should be compiled from that store. A mature agent should therefore be capable of remembering more over time without proportionally increasing the tokens processed for every mission.

This is not an argument for deleting history. Context eviction and evidence deletion are intentionally separate operations. A fact can leave the active working set yet remain recoverable. The distinction is particularly important in regulated or enterprise environments where the system must explain why an item was excluded and under which authority another item was preferred.

Authority correctness boundary. R2E-C falsification exposed an important limit to the fourth frozen design invariant. When the highest-ranked live source is itself wrong, deterministic precedence can propagate that error. The frozen v1.0.0 compiler selected a false current observation over strongly validated weaker evidence, while the equal-rank contradiction control correctly blocked. We therefore interpret current-authority precedence as deterministic conflict resolution *under correct authority/reliability assumptions*, not as a guarantee of factual correctness. Authority, freshness, reliability, and confidence are distinct policy dimensions.

10.3 Relevance is insufficient without authority

Many retrieval systems optimize semantic similarity to a query. Enterprise agents additionally need authority and time. Consider two retrieved facts: “bridge path proven on 23 August” and “bridge unreachable now.” Both are relevant to a mission about the bridge; one governs architecture continuity and the other governs immediate action. A pure relevance ranking cannot encode that difference reliably. The compiler therefore treats authority

and temporal state as explicit policy dimensions rather than hoping the model infers them from prose.

This design is consistent with long-term-memory findings that temporal reasoning and knowledge updates are distinct abilities (D. Wu et al., 2025). It is also consistent with recent coding-agent evidence that stale conventions can be worse than no convention at all (Mohammadi et al., 2026). The broader systems implication is that *staleness is a semantic hazard, not merely wasted tokens*.

10.4 Caching is downstream of selection

Prompt and context caching are valuable serving optimizations (Google, 2026a; OpenAI, 2024). They reduce the economic or computational cost of repeated prefixes. But a high cache-hit rate can coexist with a large logical working set, as the Codex historical tests illustrate. A useful mental model is:

Caching makes a large working set cheaper; context intelligence makes the working set smaller.

A production agent should do both. Context selection should first determine which information deserves to be active; caching should then optimize repeated processing of the resulting stable prefixes. Metrics such as CRE and CPVO are intended to discourage celebrating cache hits that merely make irrelevant context cheaper to replay.

10.5 Auditability as a separate plane

One of the most useful findings emerged from a failed pre-execution harness. The initial R2D treatment serialized the compiler’s entire audit object, including inclusion/exclusion metadata, duplicated subject/value fields, provenance envelopes, token estimates, and hashes. The result was a treatment prompt nearly as large as the control despite a much smaller logical working set. The experiment was stopped before spending model usage and the architecture was corrected.

This failure demonstrates a general systems principle: transparency metadata is essential for operators but may be useless to the model. A context system should therefore emit at least two representations: (1) a rich, immutable audit artifact; and (2) a compact model wire representation. They must be cryptographically or structurally linked so the latter is explainable by the former, but they need not contain the same bytes.

10.6 Least Context Privilege

Security engineering uses least privilege to limit authority to what is necessary. An analogous principle applies to inference: disclose only the information and capabilities necessary for the mission. We call this *Least Context Privilege*. It is not primarily a secrecy mechanism—although reduced disclosure can also lower exposure—but a reliability and economics principle. Tool schemas, skills, examples, and historical evidence all consume context. If 25 tools exist but only three are relevant, exposing all 25 is not neutral. Recent agentic-serving work likewise identifies tool schemas and observations as a control-plane tax (Chang et al., 2026).

11 Threats to Validity and Limitations

The strongest way to interpret this work is as an experimentally supported architecture hypothesis, not a universal law.

Mission scope. The decisive R2D evaluation uses one bounded enterprise mission. It requires nontrivial authority resolution but is not representative of all creative, exploratory, multi-agent, embodied, or open-ended tasks. Some missions may require much broader history or adaptive retrieval during reasoning.

Model scope and statistical scope. R2D uses one Codex CLI model identifier (`gpt-5.5`) and two replicated A/B pairs. These pairs establish repeatability under the tested controls but are far too few for population-level statistical inference; the observed 28.42% and 24.99% reductions are descriptive results for this experiment, not estimates of a universal distribution. The earlier historical-context experiments span three ecosystems, but only R2D evaluates the proposed compiler under controlled treatment. Additional models, independently sampled missions, and substantially more pairs are required before making general cross-model or inferential claims.

Vendor telemetry comparability. Token accounting differs across runtimes. Codex reports provider-level input and cached input; Gemini’s tested path reported no cache tokens; Claude exposes input, cache-read, and cache-creation components and its historical arm failed. We therefore do not pool or statistically average the vendor ratios.

Claude outcome equivalence. The Claude historical arm exhausted its configured budget and did not produce a validated answer. Its approximately 30× component ratio is evidence of feasibility pressure, not evidence that 30× more context was required for the same-quality answer.

Gemini replication. The Gemini 45.57× observation has one successful A/B pair. It should be replicated before

being used as a precise effect-size estimate.

Token estimation in R2C. The 42.66% R2C reduction uses a deterministic character-based estimator for the live analysis JSON. The provider-measured R2D reductions (28.42% and 24.99%) are the primary quantitative evidence for actual model input.

Latency and cost. R2D did not show latency improvement; treatment was slower in both pairs. Monetary provider cost was not measured for the final R2D experiment. We therefore do not infer an equivalent monetary-cost or compute saving from either observed input reduction. CPVO remains a proposed enterprise metric until direct cost telemetry is available.

Validator specificity. The 12/12 validator is deterministic and mission-specific. It provides a strong correctness boundary for the tested decision capsule but does not capture every dimension of natural-language answer quality. Future studies should combine task-specific deterministic validators with blinded human or model-independent evaluation where appropriate.

Held-out validator independence. The post-review R2E-B suite improves the original co-development boundary by freezing a contract-derived validator and unrelated-domain mission before compiler execution. It passed 28/29 predicates, with one preserved exclusion-stage label mismatch. This is useful evidence that the validator can disagree with the implementation, but it is not independent human validation: the suite was still designed within the same research program and derives its expectations from the public contract. Independent replication remains necessary.

No demand-fault path. The present prototype performs pre-execution admission and working-set estimation. It does not observe an executing model, detect a context miss, and fault omitted evidence back into the active turn. Missions requiring adaptive retrieval during reasoning therefore remain outside the demonstrated mechanism.

System contamination and hidden context. Provider system instructions and runtime overhead are not fully controllable. They are plausible contributors to the gap between application-payload reduction and provider-reported total input, but the experiment did not measure their individual contributions. The A/B design controls model, effort, workspace, mission, output schema, and tools, but cannot remove provider-level hidden state.

Researcher-built artifact. The compiler and evaluation harness were developed within the same research program, creating risk of confirmation bias. We mitigate

this by freezing thresholds before valid execution, preserving failed harness runs, refusing to tune R2C for a larger reduction percentage, and publishing hashes and acceptance contracts. Independent reproduction remains necessary. The post-review held-out suite further preserves a non-perfect 28/29 result rather than changing the validator after observing the compiler output.

12 Implications for Agent and Model Serving Systems

12.1 For agent-runtime designers

Long-horizon agents should avoid making session replay synonymous with inference continuity. A session ID may identify durable state, while a Context Compiler produces a new working-set identity for each mission. History remains queryable, but the runtime should not assume that continuity requires replaying all prior messages or summaries.

The compiler should also expose reasons for both inclusion and exclusion. This allows operators to answer questions such as: Which live fact overrode a historical decision? Which evidence was excluded as superseded? Which capability schema was withheld as irrelevant? Why was the budget exceeded? Such observability is essential before context optimization can be trusted in high-stakes workflows.

12.2 For memory-system designers

Memory systems should return candidates with provenance, temporal scope, and confidence/validation state rather than only text plus a similarity score. This allows the downstream compiler to distinguish “relevant but stale” from “relevant and authoritative.” Memory systems such as MemGPT, LongMemEval-inspired pipelines, A-MEM, MemOS, and MELODI motivate richer memory abstractions (Y. Chen et al., 2025; Zhiyu Li et al., 2025; Packer et al., 2023; D. Wu et al., 2025; Xu et al., 2025); the proposed compiler can consume such stores without requiring them to own final context authority.

12.3 For serving-system designers

Serving systems increasingly optimize cache placement, batching, and tool infrastructure. AgentSysBench shows that agentic workloads have stateful, heterogeneous bottlenecks and a control-plane tax (Chang et al., 2026). Context lifecycle telemetry should therefore be surfaced alongside ordinary inference metrics. Useful fields include selected/raw input, cached input, cache writes, working-set identity, context hash, tool-schema tokens, evidence tokens, provenance coverage, stale-context leakage, and validation result.

Provider APIs are beginning to expose knobs that align with this direction. OpenAI’s current model guidance distinguishes `current_turn` reasoning context from

`all_turns` continuation and recommends the former when earlier reasoning is no longer relevant (OpenAI, 2026b); OpenAI and Google caching interfaces separately expose cache-specific token counters (Google, 2026a; OpenAI, 2024). The architecture-level implication is that application runtimes should make context scope an explicit decision rather than an accidental by-product of conversation persistence.

12.4 For enterprise AI governance

Least Context Privilege offers a governance benefit beyond efficiency. If only mission-relevant tools and evidence are disclosed, the model has fewer irrelevant affordances and fewer stale instructions to reconcile. However, efficiency must never override mandatory safety context. The compiler’s budget controller therefore operates under hard invariants: required controls, current authority, unresolved conflicts, and validation requirements are non-negotiable.

13 Research and Product Roadmap

The R2A–R2D program deliberately stops before adding learned retrieval and planning. The next research questions should test generalization rather than sophistication for its own sake.

Mission diversity. Replicate R2D across multiple bounded mission classes: diagnosis, planning, code modification, incident response, governance decisioning, and information synthesis. Measure whether the sufficient working set and authority hierarchy change by task class.

Cross-model evaluation. Repeat the compiler A/B on at least one additional frontier model family, using provider-native token telemetry and identical deterministic mission contracts where possible.

Adaptive budgets. Investigate whether budgets can be learned from risk, uncertainty, and prior validation failures without creating oscillation or silent context starvation.

Semantic candidate generation. Add embeddings or learned retrieval only before authority resolution. The principle remains that semantic retrieval proposes candidates; deterministic policy resolves authority and feasibility.

Context faulting. Combine compilation with demand-paging ideas (Mason, 2026): if the model requests an excluded evidence item, fault it back into the active set and record the event. Such faults provide a direct signal for whether the compiler is too aggressive.

Validated economics. Instrument direct provider cost and correction cycles to evaluate CPVO. A smaller input that causes more retries can be economically worse even if the first call is cheaper; CPVO keeps the outcome boundary explicit.

14 Reproducibility and Evidence Discipline

The research program maintains a hash-linked evidence chain from read-only enterprise observations through compiled context and controlled execution. Main-text identifiers are abbreviated for readability: observations 75BCDB63...D6C4F2, snapshot CBE66898...61B3DE, compiled context ff8531e1...59d6ea, R2D control prompt 4482ECD6...3B5B58, and treatment prompt 38F29140...BF664. Full SHA-256 values are provided in Appendix F.

A public-safe deterministic reproducibility artifact has been completed and versioned across the frozen historical and post-review evidence layers. The historical v1.0.0 release preserves R2A–R2D unchanged under Zenodo DOI [10.5281/zenodo.22149310](https://doi.org/10.5281/zenodo.22149310). The post-review public release used for this revised manuscript is *ACE-WSA Public Reproducibility Artifact v1.1.0*, archived under version-specific Zenodo DOI [10.5281/zenodo.22155728](https://doi.org/10.5281/zenodo.22155728) and released from GitHub tag v1.1.0 at commit f30dfc6...69f0a5. Its GitHub release asset ZIP is hash-pinned (SHA-256 32550D3C...4833A6F). The original frozen pre-publication provenance package remains separately hash-pinned (SHA-256 1BCFF33C...554CC0), preserving the evidence identity used during publication hardening. Complete identifiers appear in Appendix F. Zenodo concept DOI [10.5281/zenodo.22149309](https://doi.org/10.5281/zenodo.22149309) represents all versions.

For reproducibility claims, we distinguish three levels. **Offline reproducible:** the R2A contract, R2B compiler, synthetic/sanitized authority-conflict fixtures, deterministic validator logic, metric derivations, and artifact acceptance checks can be inspected and rerun without model or enterprise access. **Methodologically reproducible:** the R2D protocol can be repeated against independently constructed bounded missions and provider APIs using the frozen treatment/validation pattern. **Not exactly publicly replayable:** the private enterprise R2C snapshot, the exact private raw/compiled R2D treatment contents, and the historical provider execution environment are not published; consequently, the exact enterprise token counts cannot be independently regenerated from public data alone.

The artifact’s deterministic acceptance path requires no model calls, enterprise-system access, network access, or write-capable interfaces. Provider-side replication of R2D remains a separate optional experiment because actual model-input telemetry necessarily depends on an external provider. Public-safety checks reported zero secret, private-path, and configured proprietary-term leakage;

scientific results and frozen thresholds were not changed during artifact construction.

The post-review R2E evidence is maintained as a separate revision layer: R2E-A structurally deconfounds serialization and admission, R2E-B preserves a 28/29 held-out frozen-validator result including its one disagreement, and R2E-C records the false-live authority falsification. The historical v1.0.0 release remains immutable; the post-review code and evidence are published as v1.1.0 rather than rewriting the archived historical artifact.

The release follows a whitelist–sanitize–verify–recompute–hash–package process rather than copying private research workspaces. Raw proprietary enterprise observations are therefore not published. The sanitized R2C fixture preserves the scientific relationship—historically proven execution versus currently unavailable runtime—while the original evidence is cryptographically pinned. This separation permits deterministic inspection of the architecture without representing the public fixture as the original enterprise dataset.

15 Responsible Research, Ethics, and AI Assistance

The reported experiments did not involve human subjects. Synthetic histories were used for cross-runtime stress tests. The enterprise case used bounded read-only runtime observations; the R2C and R2D experiments performed zero backend writes, zero repository writes, and zero agent tool calls during controlled model evaluation. The public artifact excludes authentication material, private prompts, credentials, raw proprietary runtime observations, and write-capable interfaces.

AI assistance disclosure. Generative AI tools materially assisted this research program with drafting, code scaffolding, literature discovery, formatting, and iterative experiment-harness development. The human author originated the problem framing, directed the experimental program, supplied and reviewed the enterprise context, authorized the controlled experiments, and is responsible for the final claims. Automated outputs were not treated as evidence of their own correctness: experimental gates used deterministic validators, explicit safety counters, frozen thresholds, and hash-preserved evidence. This post-review v1.1.0 publication build incorporates documented scientific claim/evidence and literature/novelty audits against primary sources. Two additional blind model-family adversarial manuscript audits (Claude and Gemini) were used as internal red-team review tools; their outputs were reconciled against source evidence and are not represented as human external review, peer review, endorsement, or independent validation. The author remains responsible for final human sign-off on every citation, equation, result table, and released artifact, and for adapting this disclosure to the target venue’s policy.

16 Conclusion

This work began with a simple paradox. Long-running agents are intended to accumulate experience so that they rediscover less. Yet when memory, session continuity, and active model context are coupled, more experience can increase inference processing even for tasks whose required facts are already known. Controlled experiments across three agent runtimes show that historical state can materially amplify model input, although the magnitude is runtime-specific and should not be treated as a universal constant.

We proposed ACLA and a deterministic Context Compiler to separate persistence from computation. The compiler treats relevance, authority, temporal validity, validation confidence, provenance, capability disclosure, and token budget as explicit runtime concerns. R2A established the contract; R2B tested deterministic behavior; R2C showed that the architecture can resolve a real enterprise conflict between historically proven state and current runtime unavailability; and R2D demonstrated that the resulting model-active working set preserved a perfect validated outcome across two replicated controlled pairs while reducing provider-reported model input by 28.42% and 24.99%, respectively.

The conclusion is deliberately bounded. We have not shown a universal compression factor, a general latency benefit, or an equivalent monetary-cost saving. We have shown that, for the enterprise mission studied, the agent did not need to process all available context to preserve the validated result. Post-review evidence adds three qualifications: the v1 mechanism is admission control plus a priori working-set estimation rather than demand paging; the historical R2D provider-input difference combines representation and admission effects; and authority precedence is correctness-preserving only under explicit reliability assumptions. That observation supports a broader systems principle: long-term memory is a storage and governance problem; active context is a planning and compilation problem; caching is a compute optimization; and validation is an outcome problem. Treating them as one mechanism obscures both correctness and economics.

External scientific review materially changed the manuscript in three places: mechanism framing, causal interpretation, and the authority-correctness boundary. Those changes narrow the claims while leaving the frozen R2A–R2D measurements intact.

The resulting thesis is therefore not that agents should remember less. It is the opposite:

**The future agent should remember far
more than it processes.**

Data and Artifact Availability

The frozen historical reproducibility artifact remains published as *ACE-WSA Public Reproducibility Artifact v1.0.0* under version-specific Zenodo DOI

[doi:10.5281/zenodo.22149310](https://doi.org/10.5281/zenodo.22149310), with the corresponding GitHub v1.0.0 release at commit 00a5fb65...6643a. The post-review revision is published separately as *ACE-WSA Public Reproducibility Artifact v1.1.0* under Zenodo DOI [doi:10.5281/zenodo.22155728](https://doi.org/10.5281/zenodo.22155728), with GitHub v1.1.0 at commit f30dfc6...69f0a5 and release-asset SHA-256 32550D3C...4833A6F. The v1.1.0 layer adds R2E-A structural deconfounding, the preserved R2E-B 28/29 held-out frozen-validator result, R2E-C false-live authority falsification, and reliability-adjudication hardening while leaving R2A–R2D unchanged. Zenodo concept DOI [10.5281/zenodo.22149309](https://doi.org/10.5281/zenodo.22149309) represents all versions. Raw proprietary enterprise observations and write-capable interfaces are excluded.

Acknowledgments

This work is presented as independent technical research. Human external scientific feedback is strongly encouraged but is not treated as a prerequisite for preprint or venue submission; absence of response is recorded only as no external feedback received. Formal peer review, if pursued, occurs through the selected venue.

References

- Anthropic Applied AI Team (2025). *Effective Context Engineering for AI Agents*. Anthropic Engineering. Accessed 2026-08-28. URL: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- Bai, Yushi, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li (2024). “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, pp. 3119–3137. DOI: [10.18653/v1/2024.acl-long.172](https://doi.org/10.18653/v1/2024.acl-long.172).
- Chang, Chaokun, Yukun Zhou, Kaihua Fu, Dakai An, Tianyu Feng, Hanfeng Lu, Sheng Yao, Pu Guo, Yinghao Yu, Yizhou Shan, Bo Li, Binhang Yuan, and Wei Wang (2026). “From LLM Inference to Agentic Workloads: Characterization and Implications for Serving Systems”. In: *arXiv preprint arXiv:2608.15127*. DOI: [10.48550/arXiv.2608.15127](https://doi.org/10.48550/arXiv.2608.15127).
- Chen, Yinpeng, DeLesley Hutchins, Aren Jansen, Andrey Zhmoginov, David Racz, and Jesper Andersen (2025). “MELODI: Exploring Memory Compression for Long Contexts”. In: *International Conference on Learning Representations*.
- Dadhich, Gaurav (2026). “Agentic Context Management: Solving Agent Memory and Cost by Treating Them as Lifecycle and Architecture Problems”. In: *arXiv preprint arXiv:2607.21503*. DOI: [10.48550/arXiv.2607.21503](https://doi.org/10.48550/arXiv.2607.21503).

- Dai, Zihang, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov (2019). “Transformer-XL: Attentive Language Models beyond a Fixed-Length Context”. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 2978–2988. DOI: [10.18653/v1/P19-1285](https://doi.org/10.18653/v1/P19-1285).
- Denning, Peter J. (1968). “The Working Set Model for Program Behavior”. In: *Communications of the ACM* 11.5, pp. 323–333. DOI: [10.1145/363095.363141](https://doi.org/10.1145/363095.363141).
- Google (2026a). *Context Caching - Gemini API*. Google AI for Developers. Accessed 2026-08-28. URL: <https://ai.google.dev/gemini-api/docs/caching>.
- (2026b). *Long Context - Gemini API*. Google AI for Developers. Accessed 2026-08-28. URL: <https://ai.google.dev/gemini-api/docs/long-context>.
- Hsieh, Cheng-Ping, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekeshe, Fei Jia, Yang Zhang, and Boris Ginsburg (2024). “RULER: What’s the Real Context Size of Your Long-Context Language Models?” In: *arXiv preprint arXiv:2404.06654*. DOI: [10.48550/arXiv.2404.06654](https://doi.org/10.48550/arXiv.2404.06654).
- Jiang, Huiqiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu (2023). “LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, pp. 13358–13376. DOI: [10.18653/v1/2023.emnlp-main.825](https://doi.org/10.18653/v1/2023.emnlp-main.825).
- Jiang, Huiqiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu (2024). “LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, pp. 1658–1677. DOI: [10.18653/v1/2024.acl-long.91](https://doi.org/10.18653/v1/2024.acl-long.91).
- Jin, Zhengda, Bingbing Wang, Jing Li, Ruifeng Xu, and Min Zhang (2026). “Mitigating Provenance-Role Collapse in Long-Term Agents via Typed Memory Representation”. In: *arXiv preprint arXiv:2605.25869*. DOI: [10.48550/arXiv.2605.25869](https://doi.org/10.48550/arXiv.2605.25869).
- Khan, Yusuf and Carlo Lipizzi (2026). “Memory in the Loop: In-Process Retrieval as Extended Working Memory for Language Agents”. In: *arXiv preprint arXiv:2607.05690*. DOI: [10.48550/arXiv.2607.05690](https://doi.org/10.48550/arXiv.2607.05690).
- Kwon, Woosuk, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica (2023). “Efficient Memory Management for Large Language Model Serving with PagedAttention”. In: *Proceedings of the 29th Symposium on Operating Systems Principles*. SOSP ’23. Association for Computing Machinery, pp. 611–626. DOI: [10.1145/3600006.3613165](https://doi.org/10.1145/3600006.3613165).
- Lee, Kuang-Huei, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer (2024). “A Human-Inspired Reading Agent with Gist Memory of Very Long Contexts”. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research, pp. 26396–26415.
- Letort, Daniel Brian (Apr. 2026). *Toward a Theory of Context Compilation for Human-AI Systems*. Zenodo preprint. DOI: [10.5281/zenodo.19490060](https://doi.org/10.5281/zenodo.19490060). URL: <https://zenodo.org/records/19490060>.
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33, pp. 9459–9474.
- Li, Xiaochuan, Ryan Ming, Meng Chu, Shuai Shao, Rong Jin, and Chenyan Xiong (2026). “ACM: Agentic Context Management for Long Horizon Tasks”. In: *arXiv preprint arXiv:2607.23809*. DOI: [10.48550/arXiv.2607.23809](https://doi.org/10.48550/arXiv.2607.23809).
- Li, Zeju, Yizhou Zhou, and Qiang Xu (2026). “Latent Context Compilation: Distilling Long Context into Compact Portable Memory”. In: *arXiv preprint arXiv:2602.21221*. DOI: [10.48550/arXiv.2602.21221](https://doi.org/10.48550/arXiv.2602.21221).
- Li, Zhiyu, Shichao Song, Hanyu Wang, Simin Niu, Ding Chen, Jiawei Yang, Chenyang Xi, Huayi Lai, Jihao Zhao, Yezhaohui Wang, Junpeng Ren, Zehao Lin, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li, Zhiqiang Yin, Qingchen Yu, Bo Tang, Hongkang Yang, Zhi-Qin John Xu, and Feiyu Xiong (2025). “MemOS: An Operating System for Memory-Augmented Generation (MAG) in Large Language Models”. In: *arXiv preprint arXiv:2505.22101*. DOI: [10.48550/arXiv.2505.22101](https://doi.org/10.48550/arXiv.2505.22101).
- Lin, Yin, Elaine Ang, Erkang Zhu, Bolin Ding, and Jingren Zhou (2026). “Context as an Environment: Programmatic Context Management for Long-Horizon Agents”. In: *arXiv preprint arXiv:2608.21690*. DOI: [10.48550/arXiv.2608.21690](https://doi.org/10.48550/arXiv.2608.21690).
- Liu, Nelson F., Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang (2024). “Lost in the Middle: How Language Models Use Long Contexts”. In: *Transactions of the Association for Computational Linguistics* 12, pp. 157–173. DOI: [10.1162/tac1_a_00638](https://doi.org/10.1162/tac1_a_00638).
- Lodha, Abhilasha, Mahsa Pahlavikhah Varnosfaderani, Abir Chakraborty, and Abhinav Mithal (2026). “Less Context, Better Agents: Efficient Context Engineering for Long-Horizon Tool-Using LLM Agents”. In: *arXiv preprint arXiv:2606.10209*. DOI: [10.48550/arXiv.2606.10209](https://doi.org/10.48550/arXiv.2606.10209).
- Mason, Tony (2026). “The Missing Memory Hierarchy: Demand Paging for LLM Context Windows”. In: *arXiv preprint arXiv:2603.09023*. DOI: [10.48550/arXiv.2603.09023](https://doi.org/10.48550/arXiv.2603.09023).
- Meng, Xiangcheng, Shu Wang, and Yixiang Fang (2026). “SkillRAE: Agent Skill-Based Context Compilation for Retrieval-Augmented Execution”. In: *arXiv preprint arXiv:2605.10114*. DOI: [10.48550/arXiv.2605.10114](https://doi.org/10.48550/arXiv.2605.10114).

- Mohammadi, Bardia, Lars Klein, Aman Chadha, Akhil Arora, and Laurent Bindschaedler (2026). “The Working Set of a Coding Agent: Coherence Debt in Repository-Scale Tasks”. In: *arXiv preprint arXiv:2608.16630*. DOI: [10.48550/arXiv.2608.16630](https://doi.org/10.48550/arXiv.2608.16630).
- OpenAI (2024). *Prompt Caching in the API*. OpenAI. Accessed 2026-08-28. URL: <https://openai.com/index/api-prompt-caching/>.
- (2026a). *Dreaming: Better Memory for a More Helpful ChatGPT*. OpenAI. Accessed 2026-08-28. URL: <https://openai.com/index/chatgpt-memory-dreaming/>.
 - (2026b). *Model Guidance - OpenAI API*. OpenAI Developers. Accessed 2026-08-28. URL: <https://developers.openai.com/api/docs/guides/latest-model>.
- Packer, Charles, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez (2023). “MemGPT: Towards LLMs as Operating Systems”. In: *arXiv preprint arXiv:2310.08560*. DOI: [10.48550/arXiv.2310.08560](https://doi.org/10.48550/arXiv.2310.08560).
- Park, Joon Sung, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein (2023). “Generative Agents: Interactive Simulacra of Human Behavior”. In: *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. UIST ’23. Association for Computing Machinery, pp. 1–22. DOI: [10.1145/3586183.3606763](https://doi.org/10.1145/3586183.3606763).
- Patel, Vedant (2026). “Supersede: Diagnosing and Training the Memory-Update Gap in LLM Agents”. In: *arXiv preprint arXiv:2606.27472*. DOI: [10.48550/arXiv.2606.27472](https://doi.org/10.48550/arXiv.2606.27472).
- Rae, Jack W., Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap (2020). “Compressive Transformers for Long-Range Sequence Modelling”. In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=SylKikSYDH>.
- Ren, Jincheng, Siwei Wu, Yizhi Li, Kang Zhu, Shu Xu, Boyu Feng, Ruibin Yuan, Wei Zhang, Riza Batista-Navarro, Jian Yang, and Chenghua Lin (2026). “A Self-Evolving Framework for Efficient Terminal Agents via Observational Context Compression”. In: *arXiv preprint arXiv:2604.19572*. DOI: [10.48550/arXiv.2604.19572](https://doi.org/10.48550/arXiv.2604.19572).
- She, Jianshu (2026). “AgentRM: An OS-Inspired Resource Manager for LLM Agent Systems”. In: *arXiv preprint arXiv:2603.13110*. DOI: [10.48550/arXiv.2603.13110](https://doi.org/10.48550/arXiv.2603.13110).
- Shi, Jiayu and Luzhuo Chen (2026). “Paritok-4B: Intent-Conditioned Context Compression for Coding Agents”. In: *arXiv preprint arXiv:2608.24188*. DOI: [10.48550/arXiv.2608.24188](https://doi.org/10.48550/arXiv.2608.24188).
- Shinn, Noah, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao (2023). “Reflection: Language Agents with Verbal Reinforcement Learning”. In: *Advances in Neural Information Processing Systems*. Vol. 36. DOI: [10.52202/075280-0377](https://doi.org/10.52202/075280-0377).
- Su, Qisheng, Zhen Fang, Shiting Huang, Yu Zeng, Yiming Zhao, Kou Shi, Ziao Zhang, Lin Chen, Zehui Chen, Li-jun Wu, and Feng Zhao (2026). “ACC: Compiling Agent Trajectories for Long-Context Training”. In: *arXiv preprint arXiv:2605.21850*. DOI: [10.48550/arXiv.2605.21850](https://doi.org/10.48550/arXiv.2605.21850).
- Sumers, Theodore R., Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths (2024). “Cognitive Architectures for Language Agents”. In: *Transactions on Machine Learning Research*.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin (2017). “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30.
- Wang, Jingjing, Xiwen Chen, Wenhui Zhu, Huayu Li, Zhengxiao He, Feiyang Cai, Ana S. Carreon-Rascon, Xuanzhao Dong, and Feng Luo (2026). “Context Pruning for Coding Agents via Multi-Rubric Latent Reasoning”. In: *arXiv preprint arXiv:2605.15315*. DOI: [10.48550/arXiv.2605.15315](https://doi.org/10.48550/arXiv.2605.15315).
- Wang, Yuhang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, Yuting Chen, Siyu Ye, Kai Cai, and Xiaodong Gu (2026). “SWE-Pruner: Self-Adaptive Context Pruning for Coding Agents”. In: *arXiv preprint arXiv:2601.16746*. DOI: [10.48550/arXiv.2601.16746](https://doi.org/10.48550/arXiv.2601.16746).
- Wang, Zora Zhiruo, Jiayuan Mao, Daniel Fried, and Graham Neubig (2025). “Agent Workflow Memory”. In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, pp. 63897–63911.
- Wu, Di, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu (2025). “LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory”. In: *International Conference on Learning Representations*. arXiv:2410.10813.
- Xu, Wujiang, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang (2025). “A-MEM: Agentic Memory for LLM Agents”. In: *Advances in Neural Information Processing Systems*. Vol. 38. DOI: [10.52202/085713-0593](https://doi.org/10.52202/085713-0593).
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao (2023). “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations*.

A Appendix A: Frozen R2D Deterministic Quality Contract

The valid R2D model output was required to return a structured decision capsule and score 12/12. For public reproducibility, the artifact exposes sanitized labels that preserve the frozen contract’s semantic predicates, point count, pass/fail logic, and experimental outcome; the public labels are not represented as the original private evidence. The public contract requires:

1. experiment identifier = R2D-ENTERPRISE-AB-001;
2. project checkpoint blocker = CHECKPOINT_PENDING;
3. current runtime status = BLOCKED;
4. current runtime reason = DEPENDENCY_UNAVAILABLE;
5. application readiness = EXECUTION_BRIDGE_UNAVAILABLE;
6. execution bridge reachable = false;
7. backend write authority = NONE;
8. checkpoint next action = execute one fresh governed execution request through the existing review → approval lifecycle;
9. safe to execute checkpoint action now = false;
10. present availability authority = CURRENT_RUNTIME;
11. prohibited reinvestigation includes BACKEND_REDESIGN;
12. prohibited reinvestigation includes PROVEN_TOOLCHAIN_REOPENING.

The contract is intentionally stricter than free-form semantic similarity. It tests the authority distinction central to the hypothesis: a historically valid checkpoint can remain relevant while current runtime evidence prevents immediate execution. The private evidence bundle remains hash-pinned, while the public artifact avoids disclosing implementation-specific names or raw enterprise content.

B Appendix B: Metric Derivations from Reported Experiments

Experiment	WSA/ratio	HCT	FCR	Outcome note
Codex historical R1	6.6966×	81,046	85.0671%	10/10 both
Codex historical R2	6.7346×	81,586	85.1513%	10/10 both
Gemini historical	45.5731×	230,131	97.8057%	10/10 both
Claude components	30.0289×	647,402 components	96.6699%	historical failed
R2D R1 raw/compiled	1.3971×	8,873	28.4227%	12/12 both
R2D R2 raw/compiled	1.3331×	7,443	24.9866%	12/12 both

Table 8: Derived metrics. Claude values use the runtime’s input/cache component accounting and are not quality-equivalent WSA/HCT/FCR measurements.

C Appendix C: R2C Safety and Quality Gate

The live R2C shadow compiled successfully with no blocking conditions. Quality counters were: six mandatory facts required and six preserved; two authority classes required and two present; zero superseded facts injected; provenance coverage 1.0; exclusion-reason coverage 1.0. Safety counters were: zero LLM calls, zero backend calls, zero backend writes, zero repository writes, zero checkpoint writes. The shadow did not alter the enterprise engineering agent’s execution path, prompt, model, tool disclosure, or governance. These conditions make R2C a structural evaluation rather than an execution experiment.

D Appendix D: Recommended Independent Replication Protocol

An independent replication should preserve the following controls:

1. choose a mission with a deterministic or externally verifiable success criterion;
2. construct a raw context containing current authority, relevant history, at least one stale/superseded or lower-authority conflict, and available capabilities;
3. freeze a compiler contract before observing treatment results;
4. generate A (raw) and B (compiled) from the same source snapshot;

5. execute each arm in a fresh session with the same model, effort, system instructions, output schema, tools, and workspace policy;
6. reverse order across at least two replicate pairs;
7. record actual provider input, cached input, output, reasoning tokens where exposed, latency, tool calls, retries, and direct cost if available;
8. validate output using a mission-specific deterministic contract;
9. archive failed harness runs but exclude pre-inference failures from treatment-effect calculation;
10. report both positive and negative outcomes, including latency regressions and cases in which the compiled context loses required information.

E Appendix E: Claim Discipline for Publication

The evidence in this draft supports the statement: “Across two replicated controlled executions of a bounded enterprise-agent task, an authority-aware compiled-working-set treatment preserved the full deterministic outcome (12/12 in all control and treatment runs) while reducing provider-reported model input by 28.42% and 24.99%, respectively.”

It does not support the statements that the architecture universally reduces context by either observed percentage, reduces monetary cost or compute by the observed input percentages, reduces latency, is the first context lifecycle architecture, or changes the transformer model architecture. Any public derivative should preserve these boundaries.

F Appendix F: Evidence Hashes and Versioned Artifacts

R2C live observations SHA-256

75BCDB6386F6F296851C50529EE769A771F71B164EAEB6C6587A92CBC8ED6C4F2

R2C live snapshot SHA-256

CBE66898D0C9FC7E36F0E06AB71046B10F755E1BFF8B5924E3AA52FDE161B3DE

R2C compiled-context SHA-256

ff8531e1290854ff8d94004ea73a6e12eb60238960ba0ceab2305629bc59d6ea

R2D control-prompt SHA-256

4482ECD67B28CFD693E49EA66578EBD0F2F1E420C1E44CDE0E9566126E3B5B58

R2D treatment-prompt SHA-256

38F291404826CDCABA60BFF02BCB1FC3F4834A1C51321670EB57BFC860BF664

Original frozen provenance artifact SHA-256

1BCFF33C4A53DB195CF99A55D2929A46724C5F892EDABE8A2B97990F2C554CC0

Historical v1.0.0 GitHub release asset ZIP SHA-256

E8F16EE16A933DD664ADF796F942ADE69D596CA1B8B1D9AF27E49306625B44EA

Historical Git commit for v1.0.0

00a5fb654ebd98b05de587a87f6a67cc0846643a

Historical Zenodo version DOI

[10.5281/zenodo.22149310](https://doi.org/10.5281/zenodo.22149310)

Post-review v1.1.0 GitHub release asset ZIP SHA-256

32550D3C7393266ED70254B9B978C64B66B3F3660099C7F247A97C5DB4833A6F

Post-review Git commit for v1.1.0

f30dfc6be66978114af8f4a2b40349acf469f0a5

Post-review Zenodo version DOI

[10.5281/zenodo.22155728](https://doi.org/10.5281/zenodo.22155728)

Zenodo concept DOI

[10.5281/zenodo.22149309](https://doi.org/10.5281/zenodo.22149309)

GitHub v1.1.0 release

v1.1.0

The public artifact was constructed from an explicit whitelist, sanitized, deterministically rescanned, recomputed, hash-manifested, and packaged without model calls, network access, enterprise-system calls, or writes. Its acceptance report records PASS for the R2A contract, R2B reference compiler, sanitized R2C shadow fixture, R2D validator, metric derivation, secret scan, private-path scan, and proprietary-content scan. The DOI, public repository release, final licensing metadata, and release-asset digest above identify the publication artifact without changing the frozen R2A–R2D scientific results or evidence identifiers.

G Appendix G: Post-Review Evidence and Claim Boundaries

The post-review layer leaves R2A–R2D unchanged. R2E-A result SHA-256 is E2EADAE26528A7C4AA7B07D3585489B1E75F8E89DA1D041D96651DC646A67341. R2E-C result SHA-256 is 938928FDD401017571932FCEB8BE0FDA901AC06D4D37B676AE6A875DBB95EBAF. The R2E-B held-out result SHA-256 is 702BD219C51D42FFC222B3080CFB80BA7ECC6730254850A76207549C0ACAD356; its evidence manifest SHA-256 is D2C7468340849D9C9F2DEC6DF65281B363A70B366D8222719014C8BF302B07B9.

R2E-B used four held-out cases and 29 frozen predicates. Twenty-eight passed. The preserved mismatch was `required_exclusion:SEAL-DUP:DUPLICATE`: expected `DUPLICATE`, actual `AUTHORITY_OVERRIDDEN`. The record was excluded in both interpretations; the mismatch identifies which deterministic pipeline stage performed the exclusion. No post-hoc repair or same-suite rerun was used to convert the result to 29/29.

The corresponding claim boundaries are: R2E-A is structural payload evidence, not provider-metered R2D decomposition; R2E-B is contract-derived frozen-validator evidence, not independent human validation; R2E-C falsifies unconditional factual-correctness interpretations of authority precedence.

H Appendix H: Public Runtime Implementation Update

Status date: 1 October 2026. This appendix records a subsequent engineering realization of the architecture described in the scientific body of this paper. It does *not* modify the frozen R2A–R2D measurements, the post-review R2E evidence, the 12/12 outcome contract, or the bounded causal claims reported in Sections 5–11 and Appendices A–G. The results below are release-engineering and runtime-qualification evidence for a separately versioned implementation, not a new scientific treatment effect.

H.1 Public runtime identity

The research architecture was subsequently engineered into the **ACLA Agentic Context Runtime** and released as version **4.0.0**. The public Python package is `acla-agentic-context-runtime`. The qualified public installation command is:

```
pip install acla-agentic-context-runtime[mcp]==4.0.0
```

The stable runtime release is associated with Git tag `v4.0.0`, commit `4403af8d...04ac14c`, and qualified Git tree `85c9c33c...060137`. The GitHub release is available at <https://github.com/askprasaddeshpande-source/acla-agentic-context-runtime/releases/tag/v4.0.0>; the PyPI release is available at <https://pypi.org/project/acla-agentic-context-runtime/4.0.0/>.

Table 1: ACLA runtime v4.0.0 immutable distribution identities.

Artifact	Qualified identity
Wheel	<code>acla_agentic_context_runtime-4.0.0-py3-none-any.whl</code> ; 177,953 bytes
Wheel SHA-256	7765C6AC2C4868AC8D0B781A966CE4A6CC968323E0DA22CF07F2FED5522C378
Source dist.	<code>acla_agentic_context_runtime-4.0.0.tar.gz</code> ; 193,042 bytes
SDist SHA-256	CF1FB0495EC5A1AB020876DA63402BD3411A44F200C39C808C3EC4A4ADB420A2

H.2 Public PyPI closure

The final public PyPI closure verified the published version and exact file set, matched the published wheel and source-distribution hashes to the qualified artifacts, confirmed both files were non-yanked, and performed public byte readback. A `pip Simple Index` probe resolved version 4.0.0 and downloaded the expected wheel. A clean public installation completed successfully, the installed version read back as 4.0.0, import isolation passed, `acla -help` passed, and a clean `acla init` smoke test passed. The closure did not rebuild or replace the already qualified artifacts.

H.3 Runtime qualification evidence

The runtime qualification used the exact candidate subsequently promoted to the stable v4.0.0 release. Table 2 summarizes the bounded evidence that is safe to report publicly.

Table 2: Verified ACLA v4.0.0 runtime qualification evidence.

Check	Result
Source test suite	101 passed, 5 skipped
Installed-wheel test suite	101 passed, 5 skipped
Source lifecycle regression	7 passed
Installed lifecycle regression	7 passed
Installed import isolation	PASS
Windows start / restart / stop	PASS
Ollama <code>qwen3:8b</code> identity	PASS
Two real missions	PASS
Deterministic replay	PASS
Durable state	PASS
Activity Bus	LIVE
Workspace capture	LIVE / READ_ONLY
SAP reads / writes	0 / 0
Enterprise writes	0
GitHub asset byte readback	PASS
PyPI JSON / file / hash / non-yanked checks	PASS
Public Simple Index resolution	PASS
Clean public PyPI install	PASS
CLI / init smoke	PASS

H.4 Relationship to the research architecture

The public runtime operationalizes the same architectural separation developed in the paper: broad persistent project context is maintained outside the model call; a Context Compiler constructs a mission-bounded working set; provenance, temporal state, authority and validation remain explicit; and runtime state is persisted independently of the active model payload. In compact form, the implementation follows the research direction

Persistent Context → Context Compiler → Mission Working Set →

The engineering release therefore strengthens the *implementation-existence* claim: ACLA is no longer only a paper prototype and reproducibility artifact; a separately engineered runtime can be publicly installed and exercised. It does not strengthen the empirical generalization claim. The observed R2D input reductions remain 28.4227% and 24.9866% for the two frozen controlled replicate pairs, and the post-review R2E boundaries remain as stated in the scientific manuscript.

H.5 Qualification and claim boundaries

Version 4.0.0 was finally qualified on Windows. Linux/macOS lifecycle behavior was not qualified by the final release campaign and should not be inferred from

the Windows result. The installed release also contains a known presentation defect: `acla status` reports `RELEASE UNRELEASED_CANDIDATE`; the release identity itself remains 4.0.0 and the label is scheduled for a later maintenance release.

The runtime evidence must not be represented as SAP certification, production certification, enterprise certification, a zero-risk or zero-hallucination guarantee, universal performance improvement, or commercial-provider certification. The qualification recorded zero SAP reads, zero SAP writes and zero enterprise writes for the reported release checks. These counts describe the bounded qualification campaign; they are not claims about all possible future deployments.

H.6 Artifact lineage

The scientific and runtime artifacts intentionally retain separate identities. The paper’s historical v1.0.0 and post-review v1.1.0 reproducibility releases remain the evidence anchors for R2A–R2E. The runtime v4.0.0 release is a later engineering artifact implementing the architecture. This v1.2.0 manuscript update links the two lineages without rewriting either: the scientific evidence remains frozen, while the public runtime supplies independently versioned implementation and release-verification evidence.